AREN SEPANIAN

BRYAN CARABALLO

Movie Recommender

Motivation:

A recommendation system to help users find movies they would enjoy. Understanding the factors that influence movie preferences allows us to build more efficient and accurate predictive models that can be used in streaming platforms

Dataset:

The Movies Dataset (https://www.kaggle.com/datasets/rounakbanik/the-movies-dataset/data)

- Consists of metadata for 45,000 movies and movies released on or before July 2017.
- Includes components like cast, crew, plot keywords, budget, revenue, posters, release dates, languages, production companies, countries, TMDB vote counts, and vote averages.
- Also contains 26 million ratings from 270,000 users for all 45,000 movies.

How it was collected:

- The data was collected from TMDB and GroupLens

Files:

- movies_metadata.csv: include posters, backdrops, budget, revenue, release dates, languages, production countries and companies.
- keywords.csv: Contains the movie plot keywords
- credits.csv: Consists of Cast and Crew Information
- links.csv: TMDB and IMDB IDs of all the movies
- links_small.csv: TMDB and IMDB IDs of a small subset of 9,000 movies
- ratings_small.csv: subset of 100,000 ratings from 700 users on 9,000 movies

Research Objectives:

1. Identify which movie components best predict preferences
    a. Such as: Genres, Keywords, Popularity, Runtime, Vote-count, Budget, Revenue, and Ratings
2. Build a simple recommender system using content-based filtering
3. Evaluate recommendation quality using similarity scores and or ratings

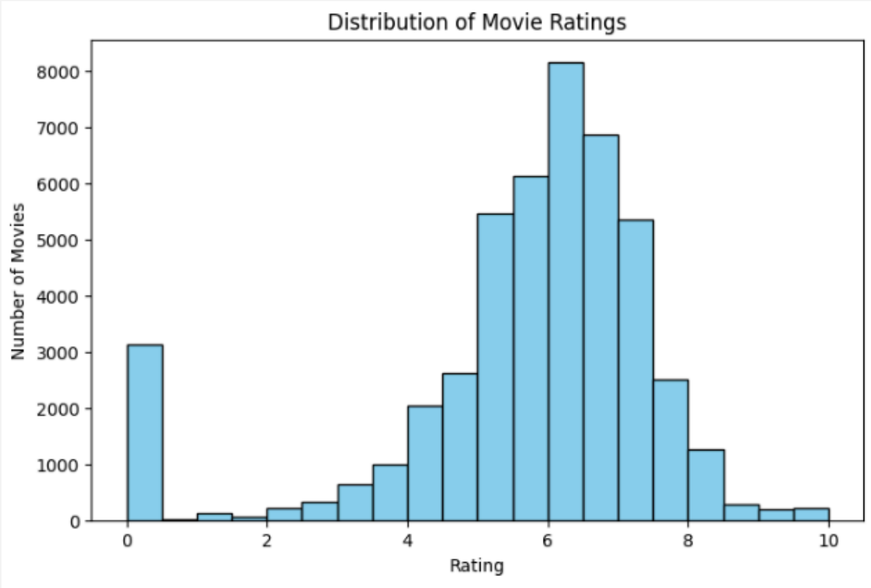
Proposed Approach:

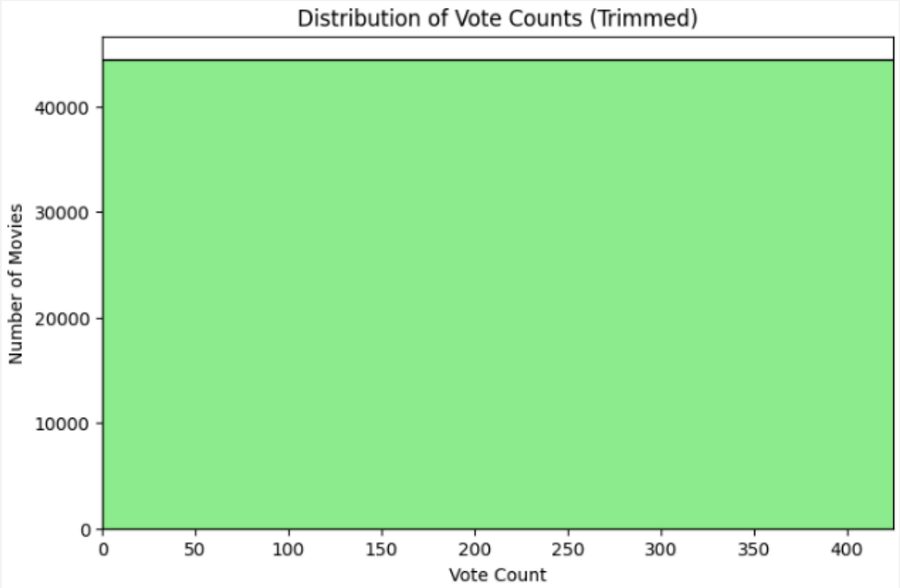
- Data Loading
- Data Cleaning
- Merging movie metadata with ratings
- Exploring genres/keywords
- Creating a content-based recommendor
- Vizualising Distributions - ratings, popularity, genre frequency, budget, revenue, and runtime

Preliminary results:

- **Data Cleaning Efforts:**
  - Cleaned and standardized the dataset by using a parsing method and then converted the rating related columns of the dataset into numerical values and filtered the movies that were above the 90th percentile in vote count to prioritize popular titles
- **Descriptive Stats:**
  - Mean movie rating: ~5.62 out of 10 (means the set has more positive ratings)
  - 90th percentile vote count = 160 (Highly rated movies)
  - Qualified movies: 4,555 (movies that have significant audience interaction)

Early Results & Visualizations





Obstacles:

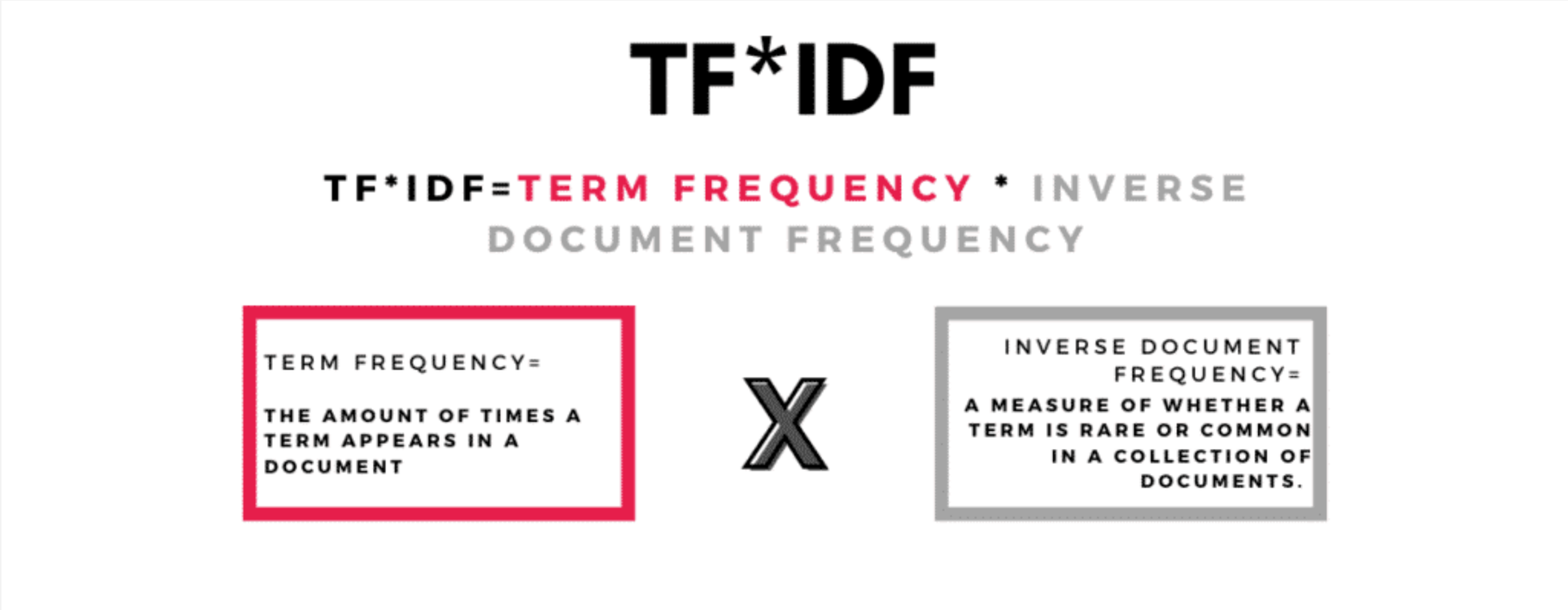
- We faced challenges, including RAM limitations from generating a 17GB cosine similarity matrix, complex JSON-formatted text fields, inconsistent movie IDs, missing or malformed data, and the need to optimize TF-IDF processing to balance accuracy with computational efficiency.
- Our cosine similarity step creates an n × n matrix (46,632 × 46,632 ≈ 2.17 billion entries).
- Stored as float64, this would need ~17 GB of RAM, but Google Colab only provides up to 12 GB, causing the session to crash.
- To move forward, we would need to either use a machine with ≥16 GB RAM or reduce the dataset size/features so the similarity matrix is smaller.

Code:

```
# Step 3: Compute cosine similarity
from sklearn.metrics.pairwise import linear_kernel

# Compute cosine similarity between all movies
cosine_sim = linear_kernel(tfidf_matrix, tfidf_matrix)

# Check shape to confirm
print('Cosine similarity matrix shape:', cosine_sim.shape)

Cosine similarity matrix shape: (45466, 45466)
```

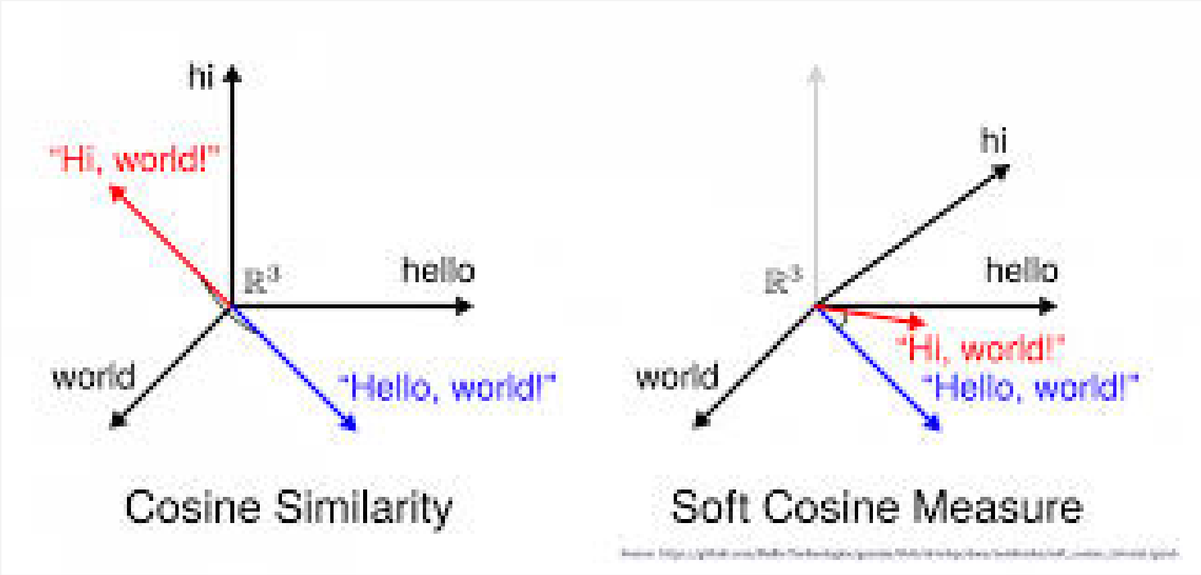
Method of Attack

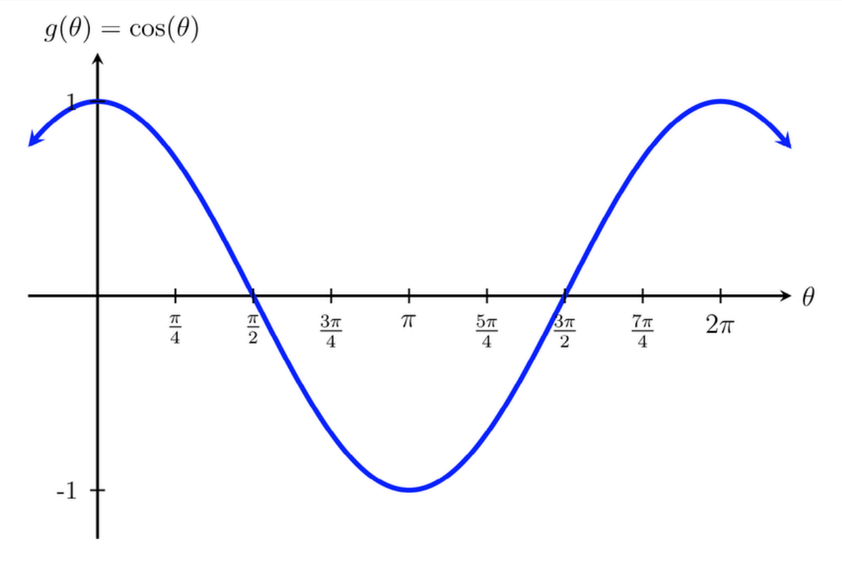
· Convert movie metadata (genres, cast, keywords) into numbers
· TF-IDF: highlights unique features, downplays common ones
· Prepare for similarity comparisons



COSINE??

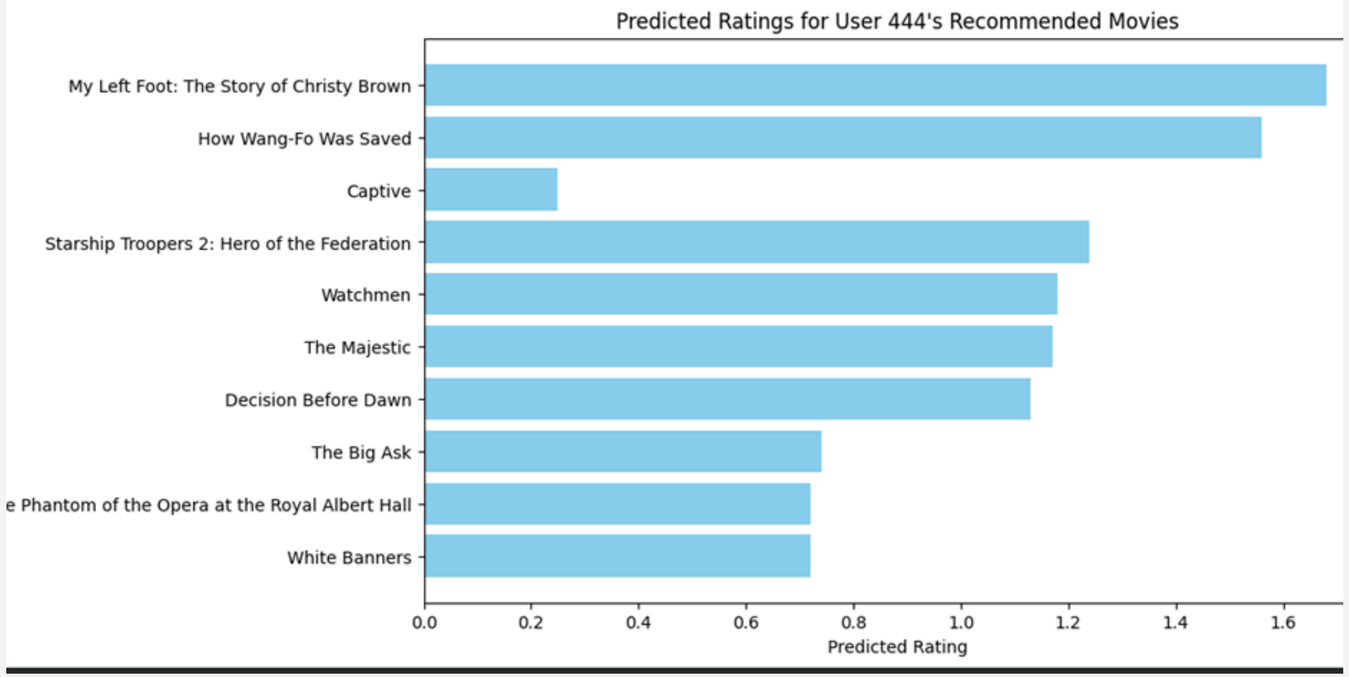
- Cosine similarity = measures closeness of movies
- Full matrix with 50k+ movies → memory crash
- Solution: compute similarity on-the-fly



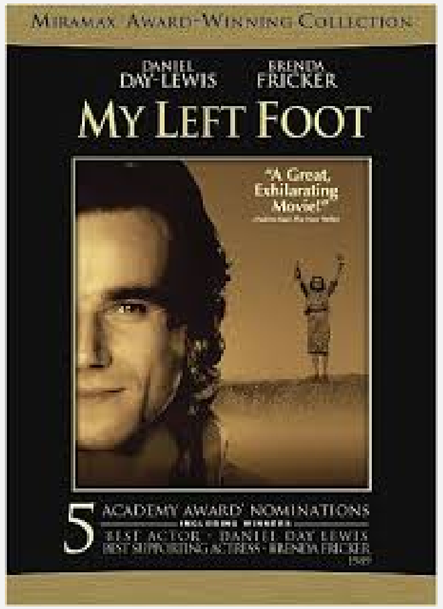


- Summary: We then measure cosine similarity — basically, how close two movies are in this feature space. At first, calculating everything at once crashed our system. So we switched to computing similarity only for the movie the user is looking at — memory stays small, results stay accurate

Just like Magic



- Use user's top-rated movie as seed
- Scale similarity → predicted rating
- Personalized top recommendations



The story highlights his struggle, his family's unwavering support (especially his mother's), and his remarkable achievements in overcoming profound physical limitations to express himself.

The story follows a humble but wise housekeeper, Hannah Thornton, who brings hope and guidance to a financially struggling family in an American town around the time of the Great Depression

Future Work: TikTok

- **Scaling**
  - If we were to scale it we could do a recommneder that the Tiktok algorithim uses we would have to go from our TF-IDF model to a neural embedding model
  - Converts text (genres, cast, plot summaries, keywords) into dense vectors — usually 128–768 numbers — that capture the semantic meaning of the content.
  - Store millions of movie vectors using FAISS for instant nearest-neighbor lookup (what powers TikTok's "For You" feed).
  - Continuously update recommendations using online learning as new data arrives.

Thank you!

Questions?